

Talking Script: Software Architecture for Essential & Accidental Uncertainty

Condensed 12-Slide Presentation Script with Timings

Total Time: 11-12 minutes

Slide 1: Title Slide

Time: 0:00 - 0:15 (15 seconds)

Good morning. I'm George Rudolph from Utah Valley University in Orem, Utah, and I'm here today to present work by Neil Harrison, Peter Aldous and myself on Software Architecture for Essential & Accidental Uncertainty.

Slide 2: The Challenge: Uncertainty in Modern Computing

Time: 0:15 - 1:00 (45 seconds)

Early computing tackled precisely specified problems with deterministic outputs. But computing has evolved dramatically—expanding to business applications, AI, control systems, and more. As this happened, inputs, outputs, and processes became more varied and less precise.

We must now deal with uncertainty in architecture and design. Current practice focuses on fault tolerance, but this misses a large part of uncertainty.

Slide 3: Essential vs. Accidental Uncertainty

Time: 1:00 - 1:50 (50 seconds)

Our work builds on Fred Brooks' distinction between accidental and essential complexity. We apply this same lens to uncertainty.

Accidental uncertainty can be separated from the problem—a transformation exists to remove it. **Essential uncertainty** must be addressed as part of the problem—no transformation can remove it.

We consider uncertainty across three dimensions: **Input**, **Transformation**, and **Output**.

Slide 4: Essential Output Uncertainties

Time: 1:50 - 2:20 (30 seconds)

This table shows the spectrum of essential output uncertainties. It ranges from problems with known right answers—like mathematical computation—to problems where the right answer can be known, like sales data. Then we have problems where the right answer exists but is infeasible to compute directly, like the game of Go. Moving along, we have unknown right answers where we can guess—image recognition. Further, effects are unknown but can be measured—optimization problems. Then systems that are chaotic—weather forecasting. No specific answer but we can observe the right direction—advertising. No single right answer because the problem is vague—automated prose writing. And finally, the problem itself is unknown—grounded theory.

Slide 5: Essential Output Uncertainties Spectrum

Time: 2:20 - 2:40 (20 seconds)

This diagram visualizes the spectrum we just discussed, showing how uncertainty ranges from deterministic problems to those where the problem itself is unknown.

Slide 6: Architectural Approaches

Time: 2:40 - 3:05 (25 seconds)

This diagram maps architectural approaches to the uncertainty spectrum. When there's virtually no uncertainty, batch processing works well. For accidental uncertainty, we use pipes and filters. For greater essential uncertainty, we need AI-based approaches.

The key is matching the architecture to the nature of the uncertainty.

Slide 7: Architecture Process for Uncertainty

Time: 3:05 - 3:35 (30 seconds)

We propose a simple process for dealing with uncertainty. First, identify uncertainties across Input, Transformation, and Output. Second, determine if each is essential or accidental. Third, can a transformation remove it? If yes, apply fault-tolerance techniques. If no, use AI-based approaches.

Slide 8: Case Study: Helicopter Rotor Balancing

Time: 3:35 - 4:20 (45 seconds)

Let me illustrate this with a real case study: helicopter rotor balancing. If the main rotor is not perfectly balanced, excess vibration occurs, causing wear, loss of power, and safety issues.

For the uncertainty analysis: The input is known measurements, but each helicopter has unique wear patterns—this is essential uncertainty. The output is measurable in IPS, and multiple solution sequences are possible. The processing initially appeared non-deterministic or chaotic.

The initial approach was to use a neural network, treating this as essential uncertainty.

Slide 9: Case Study: Helicopter Rotor Balancing (continued)

Time: 4:20 - 5:00 (40 seconds)

Further analysis revealed something interesting. Each helicopter has a unique wear profile. The initial neural network was trained on many helicopters, creating a generic approximation. But the adjustment and response history reveals unique profiles. The key insight: There exists a deterministic, repeatable sequence of adjustments with feedback loops.

What appeared essential was actually accidental, solvable with the right approach.

The results are impressive. The system reliably balances main rotors to 0.05 IPS—that's ten times better than the 0.5 IPS standard. It's actively marketed to the US Army.

Slide 10: Case Study: Online Bookstore Evolution

Time: 5:00 - 5:35 (35 seconds)

Let's look at another case study: an online bookstore. Initially, inputs and outputs were well-specified, with numerous accidental uncertainties like unreliable networks. The architecture was straightforward.

But then they added a “related books” feature. Which books are related? This is an essential uncertainty—the output cannot be known beforehand, and there's no single right answer. The approaches become probabilistic, requiring recommendation algorithms.

The evolution continued with using browsing history for promotions. This introduces even more uncertainty and requires machine learning approaches, mapping to different positions on the uncertainty spectrum.

Slide 11: References

Time: 5:35 - 5:45 (10 seconds)

Our work is published in HICSS 2026, and we also presented related work at the Intermountain Conference on Engineering, Technology and Computing in 2025.

Slide 12: Thank you!

Time: 5:45 - 11:00 (5:15 minutes for summary and Q&A)

Thank you for your attention. To summarize: We've presented a framework for understanding essential versus accidental uncertainty in software architecture. This distinction is crucial for selecting appropriate architectural approaches. Essential uncertainty requires probabilistic and AI-based methods, while accidental uncertainty can be addressed with traditional fault-tolerance techniques.

Questions?